Spark Caching and Distributed Execution

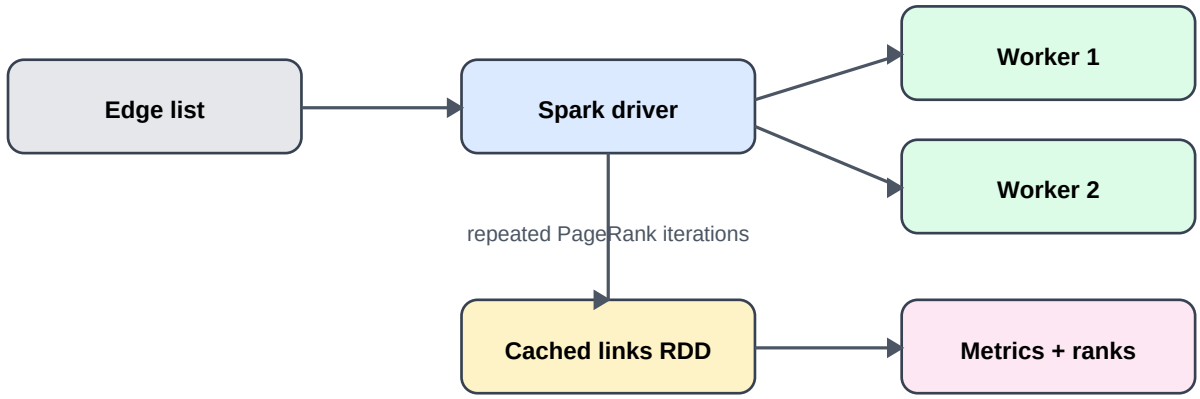
CISC3010 individual project | Project 8 | Final report baseline

Research question

When an iterative Spark application reuses the same input RDD, how much does caching change execution time, and when can cache materialization dominate the benefit?

1. Architecture

The application uses one Spark driver and two local worker processes in `local-cluster[2,1,1024]`. The driver reads a deterministic directed graph, groups outgoing neighbors into a shared `links` RDD, and repeats PageRank. Each iteration joins the links RDD with current ranks, emits contributions, reduces by destination node, and materializes the new rank vector.



The cached configuration persists `links` with `MEMORY_ONLY`. The uncached configuration uses the same lineage without persistence. Both configurations use two workers, one core per worker, four partitions, identical input seeds, and identical iteration counts.

2. Implementation and correctness

The PageRank recurrence is `rank_new(v) = 0.15/N + 0.85 * sum(rank(u)/out_degree(u))`. A sequential reference implementation uses the same recurrence. The verifier compares every node and reports the maximum absolute error. On the six-iteration demo, both cached and uncached outputs passed with a maximum absolute error of approximately 5e-13 and checksum 1.0.

3. Experiment design

The experiment uses three deterministic graph sizes, six iterations, four partitions, two local workers, and two repetitions per cache setting. The first `count()` on the shared links RDD measures initial materialization separately. The following five timings measure subsequent reuse. Raw JSON and CSV files preserve the evidence used to produce the figures.

Factor	Controlled choice	Reason
Workload	2,000 / 10,000 / 30,000 edges	Tests reuse as data grows
Cache	Off versus MEMORY_ONLY	Isolates persistence effect
Iterations	6 per run	Separates first materialization from reuse
Resources	2 workers, 1 core each, 4 partitions	Same limit for both conditions
Repetitions	2 per condition	Reports measurement variation

4. Results

Times are seconds and show mean +/- standard deviation across two repetitions. The cached run pays a higher initial materialization cost, but its later iterations become relatively cheaper as the graph grows.

Edges	Cache	Materialize	Later iter.	Total
2,000	Off	5.753 +/- 0.361	4.265 +/- 0.231	36.397 +/- 1.968
2,000	On	6.782 +/- 0.119	4.333 +/- 0.020	38.281 +/- 0.270
10,000	Off	5.932 +/- 0.074	4.450 +/- 0.046	38.044 +/- 0.201
10,000	On	6.772 +/- 0.059	4.427 +/- 0.089	38.559 +/- 0.448
30,000	Off	6.238 +/- 0.263	4.843 +/- 0.037	40.680 +/- 0.618
30,000	On	7.166 +/- 0.232	4.585 +/- 0.263	40.385 +/- 2.108

The cached subsequent-iteration speedup was 0.984x on the small workload, 1.005x on the medium workload, and 1.056x on the large workload. Caching therefore helps later reuse most clearly on the largest input, but it does not guarantee lower end-to-end runtime when the materialization cost is large relative to the workload.

5. Visual evidence

The application prints the Spark UI URL, iteration timings, partition count, storage level, and result checksum. The submitted raw CSV can be traced directly to the figures below.

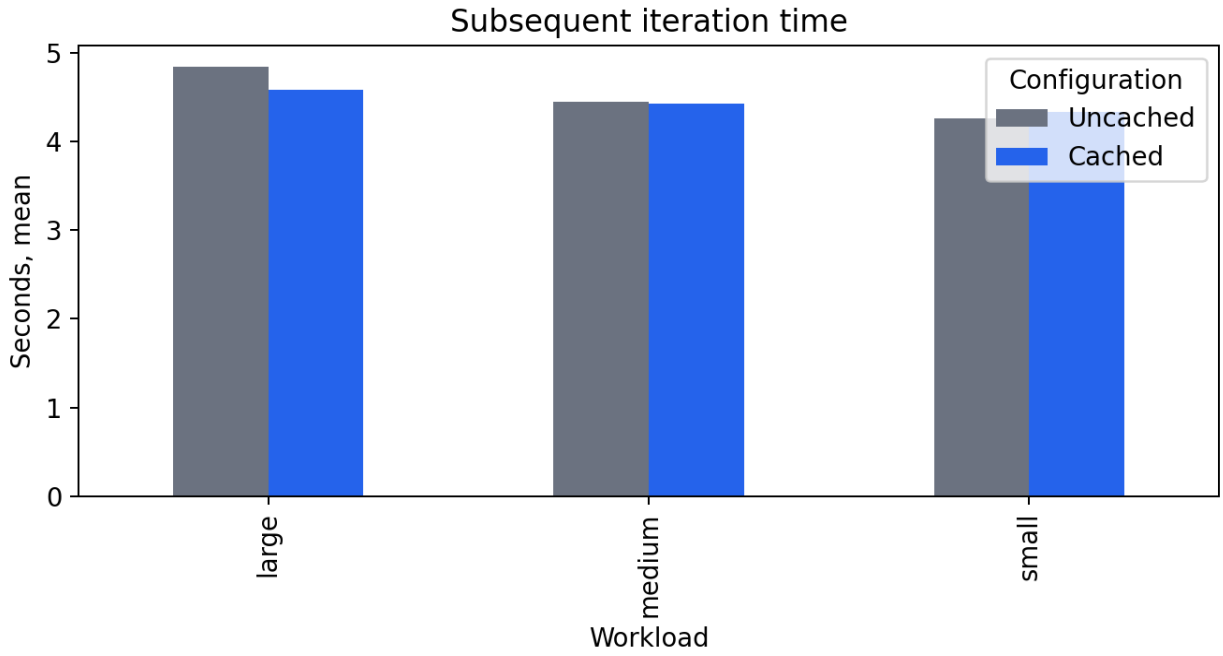


Figure 1. Mean time of subsequent iterations.

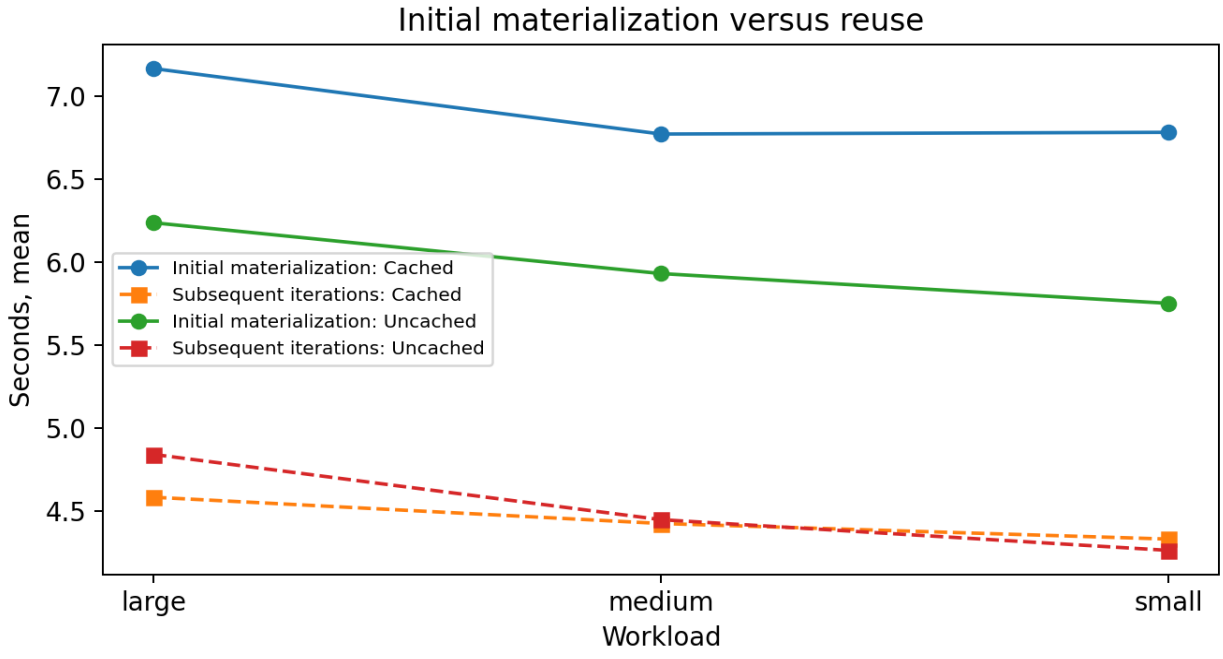


Figure 2. Initial materialization versus later reuse.

6. Connection to the reading

The project applies the RDD design idea from Zaharia et al. (2012). Persistence stores computed partitions for reuse, while lineage preserves the information needed to recompute a lost partition. This project simplifies the paper's cluster-scale fault model to two local workers and focuses on timing, cache blocks, stages, and correctness rather than throughput at scale.

7. Interpretation and limitations

The measurements support the mechanism: the cached run pays for materialization once, then avoids recomputing the shared links lineage during later iterations. The small workload shows a slowdown because Spark startup and cache materialization dominate. The medium workload is effectively a tie. The large workload shows a 5.6% improvement in subsequent iterations and a small end-to-end improvement, with higher variation in the cached total time.

All workers share one laptop, so these results demonstrate distributed execution and RDD behavior rather than multi-machine cluster performance. The workload is static and uses RDDs rather than DataFrames. The optional executor-failure and skew experiments remain outside the core scope. The metadata in the JSON output explicitly records the local master, partition budget, and checksum so that these limitations are visible.

8. Post-presentation changes

The presentation is scheduled for 26 October 2026. This report was prepared before that presentation, so the final submission must append the actual feedback and the code or documentation changes made in response. The current repository already contains the additions to the Spark example: a deterministic generator, cache toggle, separate materialization timing, sequential verifier, repeated experiment runner, raw-result retention, and figure-generation scripts.

9. AI-use statement

AI coding assistance helped with scaffolding, debugging suggestions, documentation wording, and test planning. I validated the submitted code by running the Spark demo, checking logs and output files, comparing against the sequential reference, and inspecting the generated figures. I cite the Spark documentation and the RDD paper, and I identify custom experiment and verification code in the repository.

References

[1] Matei Zaharia et al. “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing.” NSDI, 2012.
[2] Apache Spark. “RDD Programming Guide.” https://spark.apache.org/docs/latest/rdd-programming-guide.html